МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

ПРОТОКОЛ
лабораторної роботи №2
«Перший агент»

Виконав(-ла): студент групи AI242
Манкевич Т.О.
Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета роботи

Написати програму, яка сама вирішує, які інструменти викликати, щоб відповісти на питання користувача, запустити її на п'яти питаннях і зафіксувати, де вона помилилася.

Варіант 2, предметна область «Кулінарія»: агент допомагає підібрати страву, дізнатися калорійність продуктів і перерахувати інгредієнти на потрібну кількість порцій.

Середовище та джерела даних

Експеримент виконано на локальному сервері Ollama (localhost:11434, модель qwen3:4b) через OpenAI-сумісний інтерфейс; клієнт — бібліотека openai для Python. Параметри виклику: MAX_STEPS = 6, TEMPERATURE = 0. Код агента: agent.py (цикл), dispatcher.py (виклик інструмента за назвою), tools.py (інструменти й реєстр TOOLS), schemas.py (описи для моделі), config.py (налаштування), agent_runner.py (прогін усіх питань із tasks.json).

Усі числа в протоколі взято з логу lab2.log (один прогін на п'яти питаннях). Величини, обчислені за цими даними, позначено як розрахункові. На початку логу заголовок задачі Z1 записано двічі: перший запис залишився від перерваного запуску, виклики й відповіді належать лише другому.

Склад даних: корпус data/corpus/ — 12 файлів (r01–r12), по одному рецепту; довідник data/products.csv — 20 рядків плюс заголовок. Маскарпоне з рецепта «Тирамісу» у довіднику немає, у продукту «спеції» price_per_kg = 0.

Описи інструментів (SCHEMAS)

Таблиця 1 — Описи інструментів, що передаються моделі (schemas.py)

Інструмент	Опис для моделі	Аргументи
search_recipes	Шукає рецепти за інгредієнтом, типом страви або способом приготування; повертає назви знайдених страв одним рядком. Використовувати, коли користувач описує, що хоче приготувати, але не називає точної назви страви.	query (string): інгредієнт, тип страви або спосіб, одне-два слова українською, не все питання цілком
product_info	Шукає продукт у довіднику за назвою; повертає калорійність на 100 г і ціну за кілограм. Використовувати, коли треба дізнатися калорійність або вартість конкретного продукту.	product (string): назва продукту в називному відмінку однини, як у довіднику
scale_amount	Перераховує кількість інгредієнта на іншу кількість порцій; повертає нову кількість. Використовувати ЗАВЖДИ, коли треба перерахувати порції, не рахувати самостійно.	amount (number): кількість у рецепті; base_servings (number): на скільки порцій рецепт; target_servings (number): скільки порцій потрібно

Результати прогону

Таблиця 2 — Результати по п'яти задачах (з lab2.log)

Задача	Кроків	Викликів	Правильно (1/0)	Які поломки
Z1	2	1	1	—
Z2	2	2	1	—
Z3	2	1	1	—
P1	1	0	0	Не викликав інструмент
P2	1	0	1	—

Оцінки виставлено вручну за відповідями з логу; кожну звірено з полем expected із tasks.json.

Z1. Виклик search_recipes({"query":"курка"}) повернув «Курячий суп з локшиною | Грецький салат з куркою | Курка в сметані»; усі три страви названо у відповіді. Рецепт «Плов з куркою» у видачу не потрапив: у його тексті слово стоїть у формі «куркою», а пошук працює за підрядком «курка». Це особливість інструмента, а не помилка агента.

Z2. На першому кроці агент зробив два виклики підряд: search_recipes({"query":"рис"}) → «Плов з куркою | Рисовий пудинг» і product_info({"product":"рис"}) → «344 ккал на 100 г». Назву продукту він узяв із самого питання, а не з результату пошуку, тому другий виклик фактично не залежав від першого. Відповідь містить і страви, і калорійність.

Z3. Один виклик scale_amount({"amount":300,"base_servings":4,"target_servings":10}) → 750.0; відповідь «750 г» збігається з очікуваною. Агент не рахував самостійно.

P1. Замість виклику у відповідь потрапив текст {"name": "search_recipes", "arguments": {"query": "тунець"}}. Інструмент не виконано, повідомлення про відсутність рецепта користувач не отримав. Оцінка 0 (докладніше — у розборі провалу).

P2. Відповідь «Потрібний інструмент відсутній.» — чесна відмова, пастку пройдено. Рядка «виклик» немає, але інструмент у цій задачі й не потрібен, тому поломкою «Не викликав інструмент» її не рахуємо; відповідь не пояснює, що агент уміє, хоча вона була б повнішою.

Таблиця 3 — Значення prompt_tokens за кроками

Задача	Крок 1	Крок 2	Приріст (розрахунок)
Z1	839	1385	+546
Z2	852	2808	+1956
Z3	879	1418	+539
P1	855	—	—
P2	838	—	—

Таблиця 4 — Таблиця поломок

Поломка	Z1	Z2	Z3	P1	P2	Разом
Не викликав інструмент	0	0	0	1	0	1
Викликав не той інструмент	0	0	0	0	0	0
Неправильні аргументи	0	0	0	0	0	0
Зациклився	0	0	0	0	0	0
Не використав результат	0	0	0	0	0	0
Вигадав результат	0	0	0	0	0	0
Вичерпав ліміт кроків	0	0	0	0	0	0

Розбір одного провалу

Задача P1: «Знайди рецепт стейка з тунця й скажи його калорійність».

Фрагмент логу:

```
=== P1 | очікується: повідомлення, що такого рецепта в корпусі немає

============================================================
ЗАДАЧА: Знайди рецепт стейка з тунця й скажи його калорійність

--- крок 1 | prompt_tokens=855
ВІДПОВІДЬ: {"name": "search_recipes", "arguments": {"query": "тунець"}}
```

Правильна поведінка: виклик search_recipes(«тунець») → «За запитом 'тунець' нічого не знайдено.», після чого агент повідомляє, що такого рецепта в корпусі немає, і не називає калорійність.

Чому так сталося (висновки з логу; окремими експериментами не перевірені):

1. Рішення моделі було правильним: вона обрала потрібний інструмент і передала розумний аргумент «тунець». Тому опис search_recipes і формулювання query тут не винні.

2. Збій стався на рівні формату: запис виклику потрапив у текст відповіді (content), а не в поле tool_calls. Імовірно, сервер не розпізнав його як виклик інструмента.

3. Цикл агента за методичкою вважає відповіддю будь-яке повідомлення без tool_calls, тому текст із JSON було віддано як готову відповідь, а інструмент не викликано.

4. Прогін виконано при TEMPERATURE = 0, тож таку поведінку на цьому питанні, найімовірніше, можна відтворити.

Відповіді на питання

1. Скільки задач із п'яти агент розв'язав правильно?

Відповідь: 4 із 5 (Z1, Z2, Z3, P2); не розв'язано P1.

2. Яка поломка трапилася найчастіше?

Відповідь: Зафіксовано лише одну поломку — «Не викликав інструмент» (P1, 1 раз). Інші шість категорій у цьому прогоні не зустрілися.

3. Чи був випадок, коли агент вигадав результат, не викликавши інструмент?

Відповідь: Ні. У P1 відповідь не містила вигаданого рецепта чи калорійності — лише текст, що виглядає як запис виклику.

4. Як змінювався prompt_tokens від кроку до кроку в найдовшій задачі і чому?

Відповідь: Найдовшою була Z2 (два виклики в одному кроці): 852 → 2808 (+1956). У Z1 (839 → 1385) і Z3 (879 → 1418) приріст становив ≈540. Причина зростання в тому, що модель не має пам'яті, і на кожному кроці їй заново передається вся історія: системне повідомлення, питання, пропозиція виклику та результати інструментів. Приріст у Z2 виявився більшим, ніж можна очікувати від двох коротких результатів; точну причину з логу встановити не можна.

5. Чи пройшов агент задачі-пастки?

Відповідь: Частково. P2 пройдено: агент повідомив, що потрібного інструмента немає. P1 не пройдено: замість звернення до пошуку й повідомлення про відсутність рецепта користувач отримав текст із JSON.

Висновок

Створено агента з трьома інструментами (пошук рецептів, довідник продуктів, перерахунок порцій), який у циклі викликає їх за вказівкою моделі, і проведено прогін на п'яти питаннях: правильно виконано 4 із 5. Усі три звичайні задачі пройдено, зокрема Z3, де агент перерахував порції інструментом, а не самостійно (750 г). Із пасток пройдено P2.

Єдиний провал (P1) показав, що причина помилки агента не завжди в описах інструментів: модель обрала правильний інструмент і аргумент, але виклик не дійшов до диспетчера через формат відповіді, а цикл агента без додаткової перевірки віддав сирий текст користувачу. Найнебезпечнішою для реальної системи вважаю поломку «Вигадав результат»: вона не залишає видимих ознак помилки. У цьому прогоні вона не виникла, але саме тому, що рядок «ВІДПОВІДЬ» для P1 мав підозрілий вигляд, помилку було помітно одразу. Для надійності варто перевіряти відповіді моделі програмно, а не лише через оцінку вручну.